

ĐỀ THI CUỐI KỲ THỰC HÀNH

Phương pháp lập trình hướng đối tượng

PHẦN 1 — CHUẨN BỊ TẠI NHÀ (40% điểm thi cuối kỳ)

Lớp: 25C11

A. Cấu trúc bài thi và cách tính điểm

Thành phần	Hình thức	Thời gian	Trọng số
Phần 1 — Chuẩn bị tại nhà	Làm tại nhà	18 giờ	40%
Phần 2 — Thi tại lớp	Mở rộng trên project Phần 1	100 phút	60%

Mỗi phần được chấm trên thang 10. Điểm thi cuối kỳ = $0.4 \times (\text{điểm Phần 1}) + 0.6 \times (\text{điểm Phần 2})$.

QUAN TRỌNG: Phần thi tại lớp sẽ mở rộng **TRỰC TIẾP** trên chính project mà bạn nộp ở phần này. Một thiết kế sạch, tách file rõ ràng và dễ mở rộng sẽ giúp bạn tiết kiệm rất nhiều thời gian trong 100 phút thi tại lớp. Hãy đầu tư nghiêm túc vào cấu trúc mã nguồn, không chỉ vào việc “chạy được”.

B. Quy định nộp bài và môi trường biên dịch

B.1 Nộp bài

- Nộp toàn bộ project dưới dạng file nén MSSV_PreFinal.zip trên hệ thống Moodle.
- Trong file nén bắt buộc có README.txt ghi rõ: lệnh biên dịch, trình biên dịch và hệ điều hành bạn đã kiểm thử.
- Hãy giữ lại một bản sao project trên máy cá nhân; bạn sẽ cần nó cho phần thi tại lớp.

B.2 Môi trường và tổ chức mã nguồn

- Ngôn ngữ C++, chuẩn C++17. Chỉ dùng thư viện chuẩn (STL), không dùng thư viện ngoài.
- Hãy nộp kèm CMakeLists.txt hoặc file solution.
- Mỗi lớp đặt trong một cặp file .h / .cpp riêng, có include guard (#pragma once hoặc #ifndef). Toàn bộ project chỉ có duy nhất một hàm main().
- Sinh viên có thể tự đặt tên biến, tên phương thức, tên lớp nhưng phải đảm bảo ý nghĩa và dễ hiểu. Các chi tiết đề không quy định cụ thể, sinh viên được tự thiết kế nhưng phải hợp lý và logic.

B.3 Xử lý các trường hợp lỗi

- Project không biên dịch được: **0 điểm**.
- Project biên dịch được nhưng lỗi khi chạy ở một kịch bản cụ thể: chỉ mất điểm của kịch bản đó, các phần còn lại vẫn được chấm bình thường.
- Rò rỉ bộ nhớ hoặc giải phóng sai (double free) sẽ bị trừ điểm ở mục tương ứng.

B.4 Liêm chính học thuật

- Nghiêm cấm sao chép mã nguồn. Nếu phát hiện sao chép, tất cả các bài liên quan sẽ bị **0 điểm**.
- Nghiêm cấm sử dụng các công cụ AI hỗ trợ (ChatGPT, Gemini, Copilot, ...).
- Giảng viên sẽ mời vấn đáp ngẫu nhiên một số sinh viên, và mời vấn đáp bắt buộc với các bài có chênh lệch bất thường giữa Phần 1 và Phần 2. Sinh viên không giải thích được mã nguồn của chính mình sẽ bị **0 điểm** cho phần liên quan.

C. Chủ đề: Hệ thống mô phỏng Đấu trường Robot Cơ giới

(Cyberpunk Mecha Arena Simulation)

Bài thi này yêu cầu sinh viên áp dụng các nguyên lý lập trình hướng đối tượng vào một dự án mô phỏng: xây dựng phần mềm điều khiển và vận hành Đấu trường Robot Cơ giới tác chiến. Lấy cảm hứng từ các tựa game như Armored Core, Titanfall hay MechWarrior, sinh viên sẽ thiết kế bộ điều khiển trung tâm quản lý các xưởng chế tạo robot, sản xuất hàng loạt các mẫu robot chiến đấu, tổ chức tiểu đội tác chiến và quản lý vòng đời tài nguyên của kỹ sư vận hành.

D. Yêu cầu chi tiết

1. Các lớp đối tượng (7.0 điểm)

1.1 Lớp cơ sở MechaEntity — Trừu tượng (0.5 điểm)

- Vai trò: lớp cha chung nhất cho mọi thực thể trong đấu trường.
- Thuộc tính: id (int), position (struct Position { float x, y; }).
- Phương thức:

```
virtual void update() = 0;           // cập nhật trạng thái / tọa độ
virtual void draw() = 0;             // hiển thị thông tin ra console
virtual int getCombatRating() const; // lớp cơ sở trả về 0, lớp con ghi đè
virtual ~MechaEntity() {}            // hủy ảo
```

Hàm hủy ảo là bắt buộc để hủy đối tượng con đúng cách khi xóa qua con trỏ lớp cha.

1.2 Lớp BattleBot — Trừu tượng, kế thừa MechaEntity (0.5 điểm)

- Vai trò: đại diện cho các đơn vị robot di động.
- Thuộc tính: armor (int — độ bền giáp), energy (int — năng lượng), owner (Operator*).

- Phương thức:

```
virtual BattleBot* clone() const = 0; // Prototype: tao ban sao sau
void move(Position newPos); // cap nhat toa do
void setOwner(Operator* op); // gan ky su so huu
int getCombatRating() const override; // = armor + energy
```

Thuộc tính owner tạo phụ thuộc vòng giữa BattleBot.h và Operator.h. Hãy dùng khai báo trước (forward declaration) class Operator; trong BattleBot.h, và chỉ #include "Operator.h" bên trong BattleBot.cpp.

1.3 Lớp AssaultBot và SniperBot — Cụ thể, kế thừa BattleBot (0.75 điểm)

- Hai loại robot tác chiến: robot đột kích và robot bắn tỉa.
- Phải triển khai tất cả các hàm thuần ảo từ lớp cha. Hàm clone() phải trả về đối tượng mới đúng kiểu, ví dụ: return new AssaultBot(*this);
- Chỉ số khởi tạo mặc định đề nghị: AssaultBot có armor = 60, energy = 40; SniperBot có armor = 30, energy = 50.

1.4 Lớp MechaFacility — Trừu tượng, kế thừa MechaEntity (0.5 điểm)

- Vai trò: đại diện cho các công trình, nhà máy tĩnh trong đấu trường.
- Thuộc tính: durability (int), owner (Operator*).
- Phương thức: virtual BattleBot* assembleBot() = 0; // chế tạo / lắp ráp robot

Vì MechaFacility kế thừa MechaEntity, mọi lớp con CỤ THỂ của nó bắt buộc phải cài đặt cả update() và draw(), nếu không sẽ không thể khởi tạo đối tượng.

1.5 Lớp AssemblyPlant — Cụ thể, kế thừa MechaFacility (1.0 điểm)

- Vai trò: nhà máy lắp ráp robot cơ giới.
- Thuộc tính: botPrototype (BattleBot* — con trỏ tới một "nguyên mẫu" robot để nhân bản).
- Phương thức:

```
void setPrototype(BattleBot* proto); // thiết lập robot mẫu
BattleBot* assembleBot() override; // gọi botPrototype->clone()
```

- Chi phí sản xuất: mỗi lần assembleBot() thành công tiêu tốn 500 lõi năng lượng (powerCores) của kỹ sư sở hữu nhà máy. Nếu nhà máy chưa có prototype, chưa có owner, hoặc owner không đủ lõi năng lượng thì hàm trả về nullptr và không sản xuất gì.

Quan hệ giữa AssemblyPlant và botPrototype là Aggregation (sở hữu yếu). Nhà máy chỉ giữ tham chiếu tới robot mẫu, KHÔNG được delete botPrototype trong hàm hủy.

1.6 Lớp ArenaEngine — Singleton (1.0 điểm)

- Vai trò: bộ điều khiển trung tâm, quản lý chu kỳ mô phỏng và tất cả thực thể.
- Thuộc tính: static ArenaEngine* instance; std::vector<MechaEntity*> allEntities;

- Yêu cầu Singleton: hàm khởi tạo private, hàm static getInstance(), đồng thời phải vô hiệu hóa sao chép:

```
ArenaEngine(const ArenaEngine&) = delete;
ArenaEngine& operator=(const ArenaEngine&) = delete;
```

- Phương thức:

```
void registerEntity(MechaEntity* e); // dang ky thuc the vao he thong
void unregisterEntity(MechaEntity* e); // go thuc the khoi he thong
void runSimulationCycle(); // mot vong: duyet allEntities,
// goi update() va draw() da hinh
```

allEntities là quan hệ Aggregation: ArenaEngine chỉ quan sát các thực thể, KHÔNG sở hữu và KHÔNG giải phóng chúng. Vì vậy có thể đăng ký cả đối tượng cấp phát tĩnh lẫn động.

1.7 Lớp Operator (1.5 điểm)

- Vai trò: đại diện cho kỹ sư / người điều khiển cơ giới.
- Thuộc tính: name (string), powerCores (int — lõi năng lượng dự trữ).
- Quan hệ Composition: std::vector<BattleBot*> bots; và std::vector<MechaFacility*> facilities;
- Phương thức bắt buộc:

```
void addBot(BattleBot* b); // them robot VA gan b->setOwner(this)
void addFacility(MechaFacility* f); // them cong trinh VA gan owner
const std::vector<BattleBot*>& getBots() const;
const std::vector<MechaFacility*>& getFacilities() const;
int getPowerCores() const;
bool consumePowerCores(int amount); // tru neu du, tra ve true/false
BattleBot* releaseBot(BattleBot* b); // TU BO quyen so huu:
// go khoi vector, dat owner = nullptr,
// KHONG delete, tra con tro cho nguoi goi
~Operator(); // delete toan bo bots va facilities
```

Việc dùng con trỏ thô thể hiện quan hệ Composition (sở hữu mạnh): Operator chịu trách nhiệm hoàn toàn về vòng đời của các Robot và Công trình do nó quản lý. Hàm hủy ~Operator() phải duyệt hai vector và delete từng đối tượng. Riêng releaseBot() là lối thoát hợp lệ khi cần chuyển quyền sở hữu ra ngoài — sẽ được dùng ở phần thi tại lớp.

1.8 Lớp BotSquad — kế thừa MechaEntity (1.25 điểm)

- Vai trò: đại diện cho một tiểu đội robot, cho phép chỉ huy cả nhóm như một thực thể duy nhất (mẫu Composite / Delegation).
- Thuộc tính: std::vector<BattleBot*> members;
- Phương thức: void addBot(BattleBot*); void removeBot(BattleBot*); và một hàm truy xuất danh sách thành viên (cần cho toán tử + ở mục 2).
- Phải kế thừa MechaEntity để được xử lý đồng nhất. Các phương thức update() và draw() phải duyệt qua members và gọi phương thức tương ứng trên từng robot con, thể hiện sự ủy quyền (delegation).

Quan hệ Aggregation (sở hữu yếu): BotSquad chỉ chứa tham chiếu tới các robot chứ không sở hữu chúng. Hàm hủy của BotSquad KHÔNG được delete members — giải tán một tiểu đội không làm các robot bên trong bị hủy.

1.9 Ghi chú về hàm draw()

Hàm draw() không nhằm mục đích vẽ đồ họa. Mục tiêu của nó là "vẽ" trạng thái đối tượng ra màn hình console dưới dạng văn bản, ví dụ: ID, giáp, năng lượng, tọa độ, năng lực tác chiến.

2. Nạp chồng toán tử (1.0 điểm)

2.1 operator<< (0.5 điểm)

Nạp chồng toán tử xuất cho lớp Operator để in thông tin kỹ sư điều khiển: tên, số lỗi năng lượng, số robot đang sở hữu, số nhà máy đang sở hữu.

```
std::ostream& operator<<(std::ostream& os, const Operator& op);
```

2.2 operator+ (0.5 điểm)

Nạp chồng toán tử cộng để hợp nhất hai BotSquad thành một BotSquad mới chứa tất cả robot từ hai tiểu đội ban đầu. Toán tử này KHÔNG được là hàm thành viên (phải là non-member hoặc friend).

```
BotSquad operator+(const BotSquad& squadA, const BotSquad& squadB);
```

Lưu ý: vì BotSquad là quan hệ Aggregation, tiểu đội hợp nhất chỉ sao chép các con trở thành viên — đây là hành vi đúng, không cần deep copy.

3. Hàm main() kiểm thử (2.0 điểm)

Kịch bản trong hàm main phải mô phỏng đầy đủ chuỗi sự kiện sau:

- Khởi tạo hệ thống: ArenaEngine được lấy qua getInstance(); chứng minh chỉ tồn tại một thực thể duy nhất bằng cách so sánh địa chỉ hai lần gọi. (0.25 điểm)
- Kỹ sư xuất hiện và xây dựng: tạo Operator, chuẩn bị các prototype AssaultBot / SniperBot, xây một AssemblyPlant và giao cho Operator (Composition). Nhà máy dùng Prototype để nhân bản robot; in ra số lỗi năng lượng còn lại sau khi sản xuất. (0.5 điểm)
- Tập hợp tiểu đội: các robot được đưa vào một BotSquad (Aggregation). (0.25 điểm)
- Đa hình qua bộ điều khiển: đăng ký BotSquad và AssemblyPlant vào ArenaEngine, sau đó gọi engine->runSimulationCycle() MỘT lần duy nhất và cho thấy update() / draw() được gọi đa hình trên các kiểu thực thể khác nhau, đồng thời được ủy quyền xuống từng robot trong tiểu đội. (0.5 điểm)
- Kiểm tra nạp chồng toán tử: dùng operator<< in trạng thái Operator, dùng operator+ hợp nhất hai BotSquad rồi vẽ tiểu đội hợp nhất. (0.25 điểm)
- Kết thúc đúng: khi main kết thúc, ~Operator() giải phóng toàn bộ robot và công trình; chương trình thoát không lỗi, không rò rỉ, không double free. (0.25 điểm)

4. Bảng điểm tổng hợp

Nội dung	Điểm	Ghi chú
1.1 MechaEntity	0.5	Trừu tượng
1.2 BattleBot	0.5	Trừu tượng
1.3 AssaultBot + SniperBot	0.75	Prototype
1.4 MechaFacility	0.5	Trừu tượng
1.5 AssemblyPlant	1.0	Prototype + chi phí
1.6 ArenaEngine	1.0	Singleton
1.7 Operator	1.5	Composition
1.8 BotSquad	1.25	Aggregation + Delegation
2. Nạp chồng toán tử	1.0	<< và +
3. Hàm main() kiểm thử	2.0	6 mục kịch bản
TỔNG	10.0	

5. Kịch bản main() tham khảo

Dưới đây là một kịch bản mẫu để sinh viên tham khảo. Bạn được phép thay đổi chi tiết miễn là minh họa đủ 6 mục ở phần 3.

```
#include <iostream>
#include "ArenaEngine.h"
#include "Operator.h"
#include "AssaultBot.h"
#include "SniperBot.h"
#include "AssemblyPlant.h"
#include "BotSquad.h"
using namespace std;

int main() {
    // --- 1. Khởi tạo ArenaEngine (kiểm tra Singleton) ---
    cout << "--- Initializing Arena Engine ---\n";
    ArenaEngine* engine = ArenaEngine::getInstance();
    ArenaEngine* another = ArenaEngine::getInstance();
    cout << "Same instance: " << (engine == another ? "YES" : "NO") << "\n\n";

    // --- 2. Tạo ký sư và các nguyên mẫu robot (kiểm tra Prototype) ---
    cout << "--- Setting up Operator and Prototypes ---\n";
    Operator op1("Master Chief", 8000);
    AssaultBot assaultPrototype(900);    // nguyên mẫu robot dot kích
    SniperBot sniperPrototype(901);      // nguyên mẫu robot bắn tỉa
    cout << op1 << "\n\n";

    // --- 3. Ký sư xây nhà máy (Composition) ---
    cout << "--- Operator builds an Assembly Plant ---\n";
    AssemblyPlant* plant = new AssemblyPlant(10);
    plant->setPrototype(&assaultPrototype);    // Aggregation: KHÔNG delete proto
    op1.addFacility(plant);                     // Composition: op1 sở hữu plant
```

```

// --- 4. Nha may san xuat robot (Prototype + chi phi nang luong) ---
cout << "\n--- Assembly Plant produces Battle Bots ---\n";
BattleBot* bot1 = plant->assembleBot();
BattleBot* bot2 = plant->assembleBot();
if (bot1 && bot2) { op1.addBot(bot1); op1.addBot(bot2); }
cout << "Power cores left: " << op1.getPowerCores() << "\n";

// --- 5. Tap hop tieu doi (Aggregation) ---
cout << "\n--- Organizing a Bot Squad ---\n";
BotSquad squad1(1);
squad1.addBot(op1.getBots()[0]);
squad1.addBot(op1.getBots()[1]);

// --- 6. Da hinh qua bo dieu khien trung tam ---
engine->registerEntity(&squad1); // ArenaEngine chi quan sat
engine->registerEntity(plant);
cout << "\n--- Simulation cycle (polymorphism) ---\n";
engine->runSimulationCycle(); // update() + draw() da hinh

// --- 7. Kiem tra nap chong toan tu ---
cout << "\n--- Testing Operator Overloading ---\n";
cout << op1 << "\n";

plant->setPrototype(&sniperPrototype);
BattleBot* bot3 = plant->assembleBot();
op1.addBot(bot3);
BotSquad squad2(2);
squad2.addBot(bot3);

BotSquad combined = squad1 + squad2;
cout << "Combined squad:\n";
combined.draw();

// --- 8. Ket thuc: ~Operator() giai phong bots va facilities ---
engine->unregisterEntity(&squad1);
engine->unregisterEntity(plant);
cout << "\n--- Exiting main(), Operator destructor will be called. ---\n";
return 0;
}

```

Chúc các bạn sinh viên hoàn thành tốt bài thi!